# Rustake

## 1 Choix techniques

Le choix de rust pour un outil de build similaire à **make** permet l'utilisation de cet outil dans des projets avec beaucoup de dépendances en offrant des performances importantes grâce au langage. De plus le contrôle de la mémoire et son système de type permet de garantir une sécurité qui peuvent faciliter les implémentations de fonctionnalités comme la parallélisation locale.
La gestion d'erreur est aussi très explicite ce qui permet la distinction des différentes raisons d'un échec pendant le build d'un projet.

### 1.1 Les hashs

L'utilisation des hashs plutôt que les timestamps permet de passer outre les problèmes que les timestamps produisent comme la détection de faux positif après la copie d'un fichier dont le contenu est similaire. Avec les timestamps une action système qui ne change pas forcement le contenu du fichier peut entraîner un rebuild d'un fichier et donc rajouter du temps.
Pour les hashs la crate **blake3** offre des performances utiles pour cet outil de build tout en maintenant une vérification sur l'intégrité des fichiers. Elle nous permet de maintenir ainsi un cache et détecter la nécessité d'un rebuild d'un fichier uniquement si une dépendance a changé.

## 2 Difficultés rencontrées

### 2.1 Ordre des dépendances

Le choix de l'ordre des dépendances dans cet outil dépend de 2 choses, la première l'ordre de build, qui doit être build pour que le second qui en dépend puisse l'être sans problème et que se passe-t-il si 2 fichiers dépendent l'un de l'autre. Pour cela l'implémentation d'un graphe de dépendance répond aux besoins grâce au **tri topologique**, il permet à la fois la détection de cycle (donc si 2 fichiers dépendent l'un de l'autre) et aussi détermine un ordre de build en fonction des dépendances des fichiers.

### 2.2 La portabilité

Pour répondre au problème de portabilité et le lancement correct d'une commande sont gérés par la fonction *string to command* qui va se baser sur le système d'epxloitation pour produire une commande à exécuter en fonction de celui-ci.

## 3 Piste d'amélioration

- Les cibles .PHONY pour déclarer des cibles virtuelles.
- La déclaration de variable pour gérer les sous-dossiers ou rendre la configuration plus flexible.
- La parallélisation locale pour améliorer la performance de l'outil.
- Un changement d'algorithme de détection des dépendances circulaires pour aller sur un algorithme comme *l'algorithme de Kahn*.
- Une interface graphique qui pourrait permettre la visualisation du graphe de dépendance produit par l'outil.